Lab: Setting Up the Environment

Estimated time 15 minutes

Objectives

After completing this lab, you will be able to:

- Understand the importance of performing a Docker run
- Create collections to organize your data along with their unique IDs and text data
- Retrieve and display a Chroma DB collection, which will include your text data associated with ID

About Skills Network Cloud IDE

Skills Network Cloud IDE (based on Theia and Docker) provides an environment for hands on labs for course and project related labs. It is an open source IDE (Integrated Development Environment).

Important Notice about this lab environment

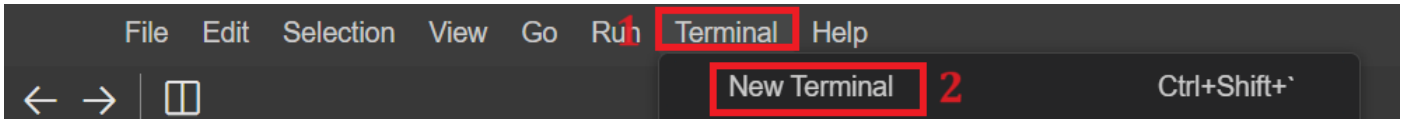
Please be aware that sessions for this lab environment are not persisted. Every time you connect to this lab, a new environment is created for you. Any data you may have saved in the earlier session would get lost. Plan to complete these labs in a single session, to avoid losing your data.

Prerequisites

- Basic knowledge of vector databases

Task 1: Setting up a Chroma DB Environment

1. In your IDE environment, select the **Terminal** tab at the top-right of the window shown at number 1 in following screenshot, and then select **New Terminal** from the drop-down menu as shown at number 2 in the same screenshot.

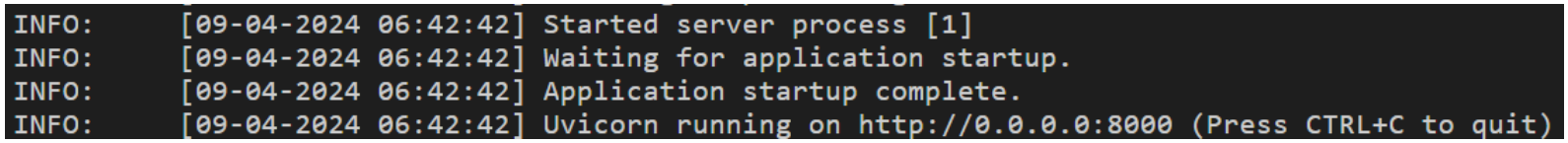


2. Run the Docker command to pull a Chroma DB image.
    - Perform docker run
        - Docker is a platform that lets developers create and run applications in the form of packages. Instead of manually downloading and installing Chroma DB, Docker helps you configure the database by pulling the image which is a pre-configured environment.
    - To create suitable environment to perform Chroma DB functionalities, paste the following command from the code block and press **Enter**.

```
docker run -p 8000:8000 chromadb/chroma
```

When you run the command `docker run -p 8000:8000 chromadb/chroma`, the Chroma database image runs in a Docker container. Let's examine the code in detail.

- `docker run` This command creates the Docker environment which is needed to pull images and then configure environments.
- `p 8000:8000` This code specifies that port 8000 on left side is for the container, which then maps to port 8000 on right side of your host machine.
- `chromadb/chroma` This instruction tells Docker which image needs to pull. In this instance, the Chroma database program in the chromadb/chroma image is in use.
- This code begins installing and running essential Docker packages. If Docker runs smoothly, then during the last phase of running this command, you will see the message "Uvicorn running on http://0.0.0.0:8000 ", as shown in the following screenshot.



Important! Please keep this terminal window open, as Docker execution will be required until the end of the lab.

Optional: Local Environment Setup

Prerequisite: Ensure that `Node.js` is downloaded and installed on your local machine.

Docker is the recommended approach for most users, as it simplifies the setup process and provides a contained environment for running ChromaDB on your local machine.

For this:

- **Install Docker:** You can download and install Docker Desktop for your operating system from the official Docker website.
- **Run ChromaDB in a Docker container:** Detailed instructions for running ChromaDB as a Docker image are available in the ChromaDB official documentation. Typically, this involves using Docker commands to pull the ChromaDB image and start a container.

Task 2: Installing the Chroma DB Environment

In this task, install the Chroma DB environment.

1. To install **chromadb** in the environment, copy and paste the following code into a new terminal window and press **Enter**.

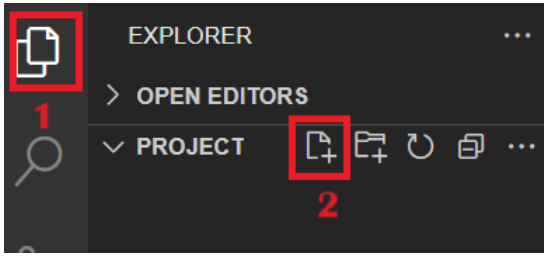
   ```
   npm install chromadb@1.9.1
   ```

   Installing the Chroma DB package will add the Chroma DB package and its functionalities to this lab.

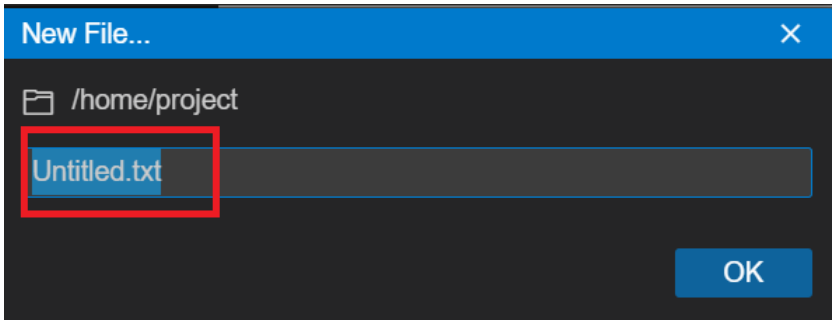
2. Then install one more dependency by performing the following command in the same terminal window where you installed the previous dependency.

   ```
   npm install chromadb-default-embed
   ```

3. Select **Explorer** on the left side of the terminal window, as shown at number 1 in the following screenshot. Then, within the **Project** folder, select **New File** as shown at number 2 in the following screenshot.



4. After performing above step, a pop-up box displays with default file name **Untitled.txt** as shown in the following screenshot. Replace its default name with `chromaData.js`.



Task 3: Create a Collection and Embed Data

After completing this task, you will be able to:

- Import and use ChromaClient to interact with the Chroma DB database.
- Structure asynchronous code using `async/await` for database operations.
- Create or retrieve collections with `getOrCreateCollection`.
- Generate unique IDs for documents, insert documents and embeddings into the collection, retrieve and log all items, and execute the process.

Your task is to create a collection and embed data.

1. First, import the ChromaClient class from the chromadb package. You will use this class to create an instance of the client that interacts with the Chroma DB database. To import the ChromaClient, you must use the required keyword of “chromadb”. You include this command in your Javascript file.

```
const { ChromaClient } = require('chromadb');  // Imports the ChromaClient class from the chromadb library
const client = new ChromaClient();             // Creates a new ChromaClient instance
```

2. Next, create the main asynchronous function. This code block also contains the code for interacting with the Chroma DB database.

```
async function main() {
    try {  // Place your asynchronous code here
    } catch (error) {
        console.error("Error:", error);  // Catches and logs any error that occurs in the try block
    }
}
```

3. Then, create a collection inside the database. For this task you will construct the collection inside the `try` block of the `main` function.

- The following code retrieves or creates a collection named "my_basic_collection" from the database using the client's `getOrCreateCollection` method. This code makes sure that a collection exists in the database, either by getting an existing collection or by creating a new collection if the collection does not exist. The collection variable holds a reference to this collection for further operations.

```
const collection = await client.getOrCreateCollection({
name: "my_basic_collection" // Specifies the name of the collection to retrieve or create
});
```

4. Next, define an array named "texts" containing sample text string data. Use the following code to construct your sample text string data.

```
const texts = [
      "This is sample text 1.", // First text item in the array
      "This is sample text 2.", // Second text item in the array
      "This is sample text 3." // Third text item in the array
];
```

5. Then, generate unique identifers, or IDs for each text item in the array. To complete this task, the following code generates unique IDs for the documents based on their index in the text array.

- Each ID has the format `document_<index>`, where `<index>` starts from 1.
- The `add` method calls the collection object to add documents to the collection.
- This coding method takes an object with IDs and document properties. Ids contain an array of document IDs, and documents contain an array of corresponding document texts.

```
// Create an array of unique IDs for each text item in the 'texts' array
// Each ID follows the format 'document_<index>', where <index> starts from 1
const ids = texts.map((_, index) => `document_${index + 1}`);
```

6. Next, add the data and IDs to your created collection.

```
// Use the 'add' method to insert documents into the 'collection'
// Each document has a unique ID from the 'ids' array and content from the 'texts' array
await collection.add({ ids: ids, documents: texts });
```

7. Finally, you can retrieve all items that are in the collection using the `get` method. The result logs to the console for verification.

```
// Retrieve all items currently stored in the 'collection'
const allItems = await collection.get();
// Log the retrieved items to the console for inspection
console.log(allItems);
```

8. To view the output, you need to call the `main` function.

```
main()
```

Next, learn how to check the output.

Task 4: Check the Output

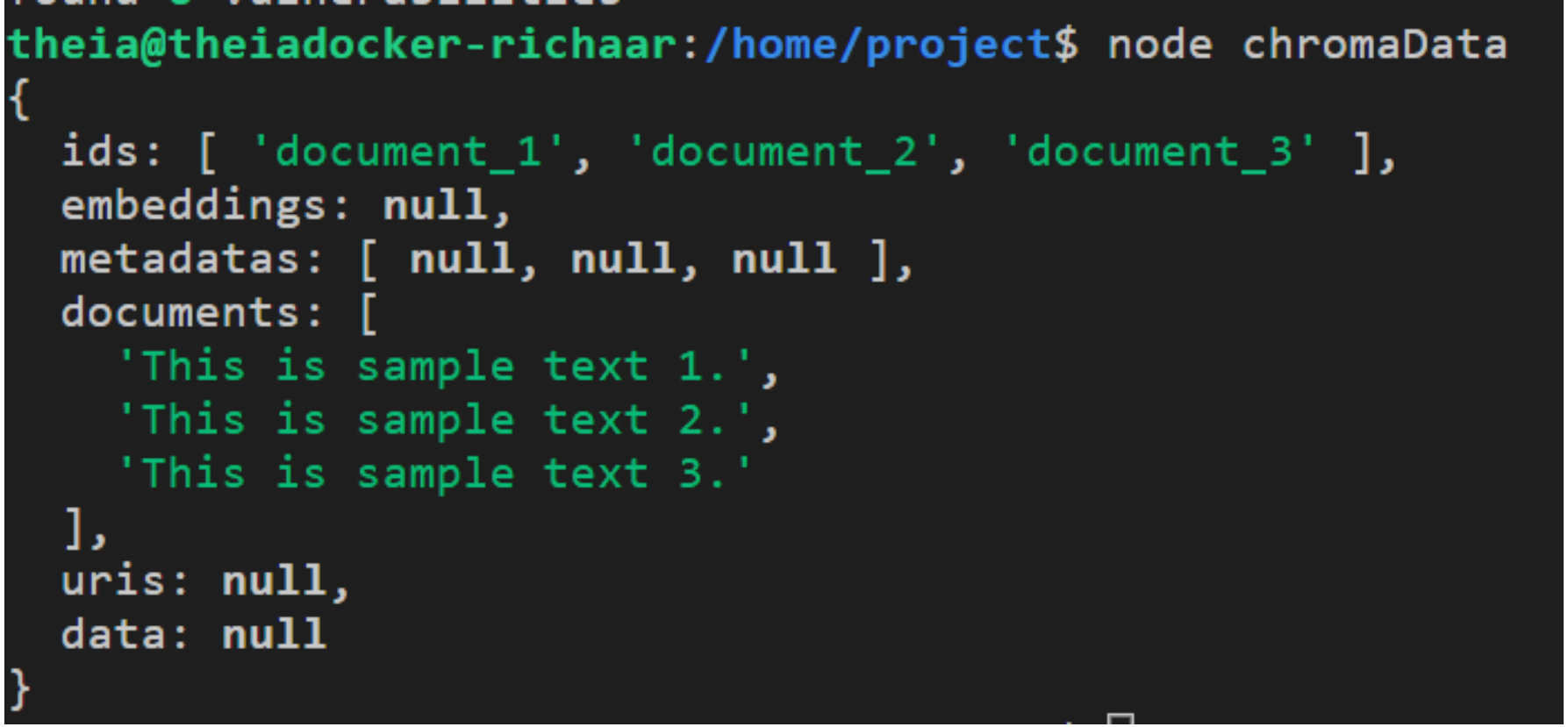
Click here to view the full solution code.

View the output so that you can verify that the collection includes the data.

1. Verify that the terminal in which you execute Docker command is still running.
2. In the terminal window, perform the given command to see the output.

```
node chromaData.js
```

To check the output, open the new terminal window. The output displays the ids and document values added in the collection of chromadb similarly to the following screenshot.



Practice Exercises

Important: Data in this lab is not persisted. If you are performing these practice exercises at a time other than when you performed the lab tasks, or you have closed the Docker window, you must to reinstall `chromadb`.

1. Create a file named `groceryData.js` where you will work with given array just like the text array in Task 3 of this lab. This file will contain grocery items including Milk, Chicken, Curd, Butter, and Bread.

   Hint: groceryitems

▶ Click here for the solution

2. Create the required Chroma DB package and create one variable to add this package functionality.

   Hint: `require` chromadb package

▶ Click here for the solution

3. Create a function named `groceryMain()`` and within this function create a collection named "groceries"“.

   Hint: Create a function using the function's keyword and use the `getOrCreateCollection` method to create the collection.

▶ Click here for the solution

4. Create IDs for each element of grocery array.

   Hint: Iterate the array to create an ID for each element.

▶ Click here for the solution

5. Then add the ids and groceryItems in the collection and get the details using built-in methods.

   Hint: Use the `add()` function.

▶ Click here for the solution

6. Get the items within a variable and display using **console.log** in terminal.

   Hint: Use `get()` function

▶ Click here for the solution

7. To view the output use the `groceryMain()` function.

   Hint: Use `groceryMain()`

▶ Click here for the solution

8. Verify that the output is correct by running the following command in terminal and reviewing the output.

```
node groceryData.js
```

Note: Make sure that the `docker run` command is still running in the other terminal.

Next, review the following code sample that contains the code used for all of the practice exercises.

Click here to view the full solution code.

Conclusion

Congratulations! You have successfully learned how to:

- Create a collection in Chroma DB using the `getOrCreateCollection` method.
- Include text data and its related ID in a Chroma DB collection.
- Use the `get` function to retrieve and display your text data associated with each retrieved document.

Author(s)

Richa Arora